SPECIFICATION

Calypso Terminal API

Storage Card

Version 2.0.0-SNAPSHOT, 2026-07-20

⚠ ***Warning. The present document cancels and replaces the previous release.***

© Calypso Networks Association 2026— Reproduction is prohibited without prior written agreement

Link and Contact



Website
https://calypsonet.org



Support
support@calypsonet.org

Copyright © Calypso Networks Association, https://calypsonet.org.

This specification is licensed under the **Creative Commons Attribution-NoDerivatives 4.0 International** *(CC BY-ND 4.0).*

You are free to **share** *— copy and redistribute this document in any medium or format for any purpose, even commercially — under the following terms:*

- ***Attribution*** *— You MUST give appropriate credit to the Calypso Networks Association, provide a link to the license, and indicate if changes were made. You MAY do so in any reasonable manner, but not in any way that suggests the Calypso Networks Association endorses you or your use.*
- ***NoDerivatives*** *— If you remix, transform, or build upon this document, you MAY NOT distribute the modified material.*

No additional restrictions — You MAY NOT apply legal terms or technological measures that legally restrict others from doing anything the license permits.

Implementation grant *— The* **NoDerivatives** *condition above applies to this specification* **document** *only — its text, tables and diagrams. It does* **not** *restrict the use of the technical information the document contains. The Calypso Networks Association hereby grants everyone an irrevocable, worldwide, royalty-free permission to use the information contained in this specification to design, develop, and distribute software implementations of this API, in any programming language, under the license of their choice.*

The full license text is available at https://creativecommons.org/licenses/by-nd/4.0/.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Table of Contents

1. Abstract . . . 5
2. Introduction . . . 6
2.1. Purpose . . . 6
2.2. Scope . . . 6
2.3. Conformance . . . 6
2.4. Document Conventions . . . 7
2.4.1. Naming . . . 7
2.4.2. Data types . . . 7
2.4.3. Type tables . . . 7
2.4.4. Operation tables . . . 8
2.4.5. Operation contracts . . . 8
2.4.6. Concurrency . . . 8
2.5. References and Resources . . . 8
2.5.1. Calypso References . . . 8
2.5.2. Normative References . . . 9
2.5.3. External Resources . . . 9
3. Glossary and Acronyms . . . 10
3.1. Glossary . . . 10
3.2. Acronyms . . . 10
4. Architectural Overview . . . 11
4.1. Functional positioning . . . 11
4.2. Logical namespaces . . . 11
4.3. UML class diagram . . . 11
5. API Specification . . . 13
5.1. Constants . . . 13
5.1.1. Storage Card API Properties . . . 13
5.2. API Interfaces . . . 13
5.2.1. Storage Card . . . 13
Get Block . . . 13
Get Blocks . . . 13
Get Product Type . . . 14
Get System Block . . . 14
Get UID . . . 14
5.2.2. Storage Card API Factory . . . 15
Create Storage Card Selection Extension . . . 15
Create Storage Card Transaction Manager . . . 15
5.2.3. Storage Card Security Settings . . . 15
5.2.4. Storage Card Selection Extension . . . 16
Prepare MIFARE Classic Authenticate With Key . . . 16
Prepare MIFARE Classic Authenticate With Key Number . . . 17
Prepare Read Block . . . 17
Prepare Read Blocks . . . 17
Prepare ST25 Read System Block . . . 18
5.2.5. Storage Card Transaction Manager . . . 18
Prepare MIFARE Classic Authenticate With Key . . . 19
Prepare MIFARE Classic Authenticate With Key Number . . . 19
Prepare Read Block . . . 20

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Prepare Read Blocks 20
Prepare ST25 Read System Block 20
Prepare ST25 Write System Block 21
Prepare Write Blocks 21
5.3. Enumerations 22
5.3.1. MIFARE Classic Key Type 22
5.3.2. Storage Card Product Type 22
5.4. Errors 23
5.4.1. Storage Card Authentication Failed 23
5.4.2. Storage Card Card Communication 23
5.4.3. Storage Card Invalid Card Response 24
5.4.4. Storage Card Reader Communication 24

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

1. Abstract

This document is the official technical specification of the **Terminal Storage Card API** standardised by the **Calypso Networks Association** (CNA). It defines, in a language-agnostic way (using a notation inspired by the Kotlin language), the interfaces, classes, enumerations, errors, structural relationships and behavioural requirements that any conforming implementation of the Terminal Storage Card API MUST satisfy.

The Terminal Storage Card API extends the **Terminal Reader API** with the abstractions required to manipulate **storage cards** (MIFARE Ultralight, MIFARE Classic 1K/4K, ST25/SRT512), all contactless cards compliant with ISO/IEC 14443. It exposes a block-oriented memory image, an authentication mechanism for MIFARE Classic and a transaction manager that batches read and write commands.

Document Status

Reference	YYMMDD-SP-CNATerminalAPI-StorageCard
Short name	CNA-TSC-API
Version	2.0.0-SNAPSHOT
Revision date	2026-07-20
Editor	Calypso Networks Association
Source repository	https://github.com/calypsonet/calypsonet-terminal-storagecard-api-spec
Reference license	Creative Commons Attribution-NoDerivatives 4.0 International (CC BY-ND 4.0)

The present document specifies the 2.0.0-SNAPSHOT of the Terminal Storage Card API. It defines a single, coherent baseline against which conforming implementations are evaluated; the **Since** column indicates the version in which each member was introduced.

Revision List

This section lists the high-level changes per version. The complete, fine-grained changelog is maintained in the `CHANGELOG.md` file at the root of the source repository.

Version / Date	Modifications
v2.0.0-SNAPSHOT 2026-07-20	Baseline.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

2. Introduction

2.1. Purpose

The Terminal Storage Card API defines the contracts required to:

- select storage cards through the **Terminal Reader API** selection mechanism,
- perform block-level read and write operations,
- expose card-specific metadata (product type, UID, system block),
- authenticate against MIFARE Classic sectors using Key A or Key B.

Conforming implementations MUST guarantee interoperability with CNA-TR-API reader implementations.

2.2. Scope

This specification covers:

- the following storage card products, all contactless cards compliant with ISO/IEC 14443 and identified by the values of `StorageCardProductType`:
  - MIFARE Ultralight,
  - MIFARE Classic 1K/4K,
  - ST25/SRT512;
- the factory used to instantiate the public types of the API,
- the abstract representation of a storage card,
- the card-selection extension used to enrich a selection scenario with storage-card commands,
- the transaction manager used to exchange APDUs with the selected card,
- the family of storage-card-specific errors.

The following topics are **out of scope**:

- the wire-level protocols specific to each storage card technology,
- the management of cryptographic key material for MIFARE Classic authentication.

2.3. Conformance

A software product conforms to this specification if and only if:

1. it implements every interface and class defined in Chapter 5 with the operations and semantics prescribed in this document,
2. it raises errors exclusively of the types defined in Section 5.4 for the situations described,
3. it preserves the structural relationships described in Chapter 4.

Throughout this specification, the key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY** and **OPTIONAL** are to be interpreted as described in RFC 2119 and RFC 8174 when, and only when, they appear in all capitals.

In conformance with RFC 2119, **SHALL** is equivalent to **MUST**; this specification uses **MUST** exclusively in order

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

to avoid ambiguity.

2.4. Document Conventions

2.4.1. Naming

Type names follow `UpperCamelCase`; operation, parameter and enumeration-member names follow `lowerCamelCase` except for enumeration values which use `UPPER_SNAKE_CASE`. Names defined by this specification are reproduced as-is in the UML class diagram (Section 4.3).

2.4.2. Data types

The following type names, whose notation is inspired by the Kotlin language, are used in operation signatures and are mapped, in each binding, to the closest equivalent native type. Implementations MUST preserve the semantic intent rather than the syntactic form.

Type	Description	Typical bindings
`String`	Sequence of characters, UTF-8 encoded by default.	`String`, `string`, `str`.
`Boolean`	Two-state truth value.	`boolean`, `bool`.
`Int`	Signed 32-bit integer.	`int`, `Int32`, `i32`.
`ByteArray`	Ordered sequence of octets.	`byte[]`, `[u8]`, `bytes`.
`Any`	Value of any type, root of the type hierarchy.	`Object`, `Any`, `object`, `Box<dyn Any>`.
`T?`	Nullable value: either a `T` value or `null`.	`Integer` (boxed), `Optional<T>`, `T?`, `Option<T>`.
`null`	Absence of value, the alternative held by a nullable type.	`null`, `nil`, `None`.
`Self`	Own type of the receiver: a fluent operation returns the instance it was called on.	`Self`, `T` (recursive generic).
`Unit`	Indicates that an operation returns no value.	`void`, `Unit`, `()`.

2.4.3. Type tables

Each interface, class, enumeration and error defined in this document is described by a table of the form:

Name	The type's normalized name.
Kind	The category of the type (interface, class, enumeration, error, etc.).
Namespace	The namespace in which the type is defined.
Extends	The type extended by this type, when applicable.
Implements	The interface implemented by this type, when applicable.
Since	The version of the API in which the type was introduced.
Purpose	A normative description of the type's responsibility.
See also	Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

The **Purpose** row states, in one or two sentences, the responsibility of the type and the way an instance is obtained. It is meant to be scanned, not read: any content that requires structure or depth — rules, lists, notes, value tables or examples — is placed as prose below the table.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

2.4.4. Operation tables

Each operation defined in this document is described by a table of the form:

Signature	The operation's signature.
Since	The version of the API in which the operation was introduced.
Description	A normative description of the operation's behaviour.
Parameters	A description of each input parameter.
Returns	A description of the returned value, if any.
Pre-conditions	The conditions the caller MUST satisfy before invoking the operation, each prefixed by its nature.
Errors	A list of error conditions a correct caller must handle, each mapped to a specific error type.
See also	Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

2.4.5. Operation contracts

To avoid repetition, the following rules apply to every operation table in Chapter 5 and are therefore not restated for each operation:

- **Pre-conditions.** The **Pre-conditions** row states what the caller MUST guarantee before invoking the operation. Violating one is a contract error, never listed in the **Errors** row, which describes only the outcomes a correct caller must handle. Each entry is prefixed by its nature, which determines the error raised when the condition is not met:
  - **Argument**—an illegal argument error. Applies to any invalid argument, including values bounded by a card or SAM protocol.
  - **Range**—an index out of bounds error. Applies when the argument designates a position within a collection or memory image exposed by the API.
  - **State**—an illegal state error.
  - **Capability**—an unsupported operation error.
- **Non-`null` results.** Unless the return type is marked nullable (`T?`) or the **Returns** row states otherwise, an operation returns a non-`null` value.
- **Fluent composition.** Builder-style operations return the current instance so that calls can be chained.

2.4.6. Concurrency

Unless explicitly stated otherwise, the stateful types defined by this specification are **not** thread-safe. A given instance MUST be accessed from a single thread at a time; concurrent use of the same instance without external synchronisation results in undefined behaviour. Distinct instances MAY be used concurrently.

2.5. References and Resources

2.5.1. Calypso References

References to CNA Terminal APIs designate the indicated **major** version; any later release within the same major is backward-compatible per that API's evolution policy.

Reference	Document
[CNA-TR-API]	*CNA Terminal API — Reader* (SP-CNATerminalAPI-Reader), version 3.0, Calypso Networks Association.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

2.5.2. Normative References

Reference	Document
[RFC 2119]	*Key words for use in RFCs to Indicate Requirement Levels*, S. Bradner, March 1997.
[RFC 8174]	*Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*, B. Leiba, May 2017.
[ISO/IEC 14443]	*Identification cards — Contactless integrated circuit cards — Proximity cards*.
[PC/SC]	*Interoperability Specification for ICCs and Personal Computer Systems*.

2.5.3. External Resources

Resource	Link
Calypso Networks Association	https://calypsonet.org
Terminal APIs website	https://terminal-api.calypsonet.org
Terminal APIs documentation	https://docs.terminal-api.calypsonet.org

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

3. Glossary and Acronyms

3.1. Glossary

Term	Definition
Block	Smallest addressable storage unit on a storage card. The block size depends on the card product type.
Sector	Group of blocks sharing the same access conditions on a MIFARE Classic card.
System Block	Block containing card-specific metadata and configuration data. Available on selected card types only.
Storage Card	Card whose primary purpose is to store binary data in addressable blocks, by opposition to a Calypso or generic application card.
MIFARE Classic	NXP MIFARE Classic 1K/4K family of contactless storage cards using sector-based authentication.
UID	Unique IDentifier of the card.
OTP	One-Time-Programmable memory area whose bits can only transition from `0` to `1`.

3.2. Acronyms

Abbreviation	Expansion
APDU	Application Protocol Data Unit
CNA	Calypso Networks Association
OTP	One-Time-Programmable
PC/SC	Personal Computer / Smart Card (interoperability standard)
SPI	Service Provider Interface
UID	Unique IDentifier
UML	Unified Modelling Language

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

4. Architectural Overview

4.1. Functional positioning

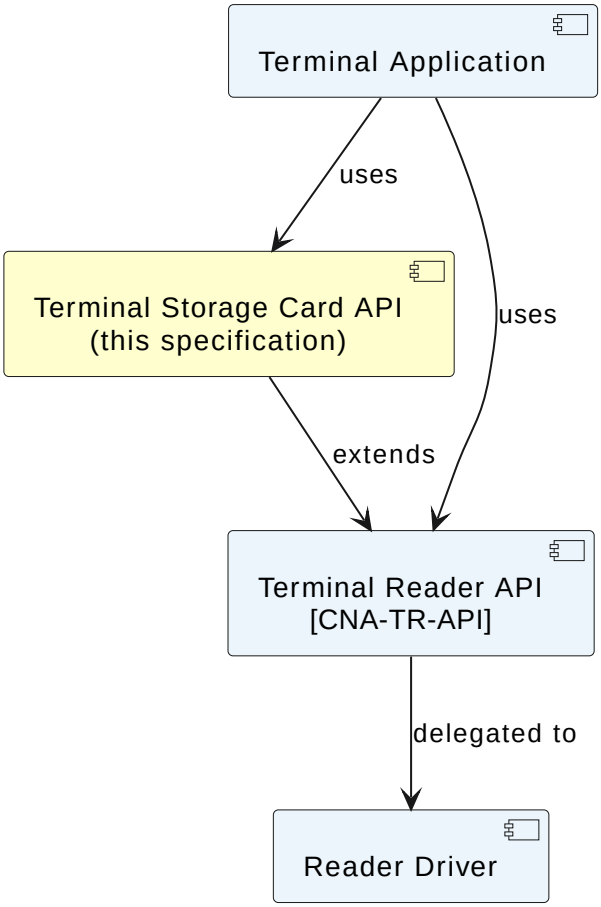


Figure 1. Terminal Storage Card API — Architecture Overview

4.2. Logical namespaces

Namespace	Role	Summary
`storagecard`	Public API	Factory, properties, key types and errors.
`storagecard.card`	Public API	Storage card abstraction, product type enumeration and selection extension.
`storagecard.transaction`	Public API	Transaction manager for storage cards and its security settings.

4.3. UML class diagram

The following UML class diagram provides a visual representation of the types and relationships defined by this specification:

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

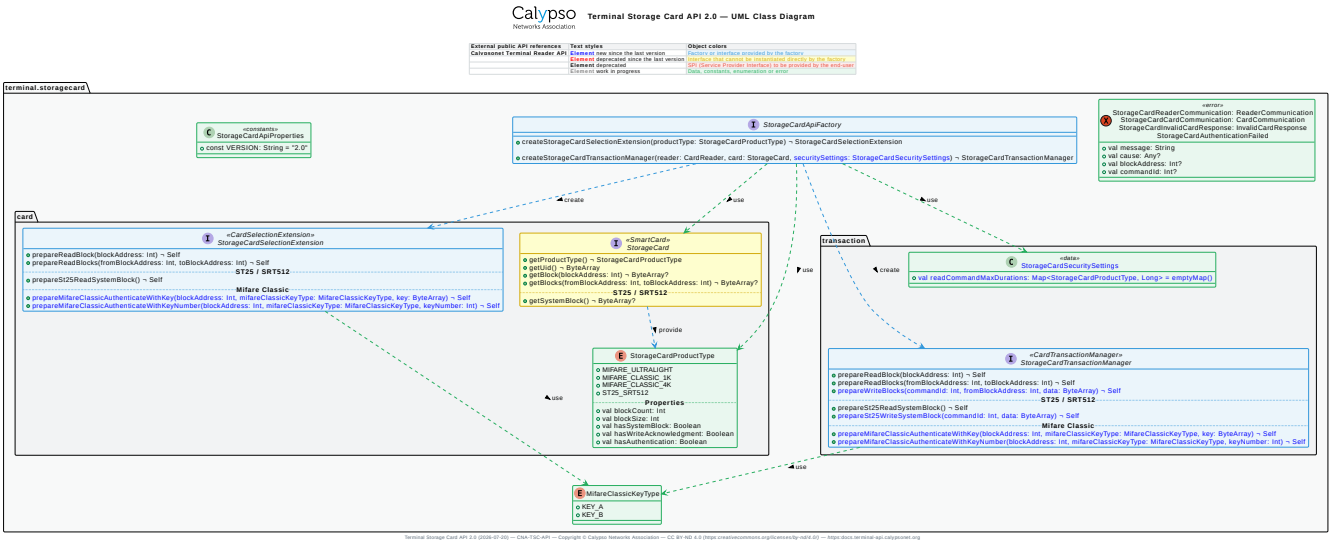


Figure 2. Terminal Storage Card API — UML Class Diagram

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5. API Specification

5.1. Constants

5.1.1. Storage Card API Properties

Name	`StorageCardApiProperties`
Kind	Constants
Namespace	`storagecard`
Since	1.0
Purpose	Exposes the immutable properties of the Terminal Storage Card API.

Name	Type	Since	Description
`VERSION`	`String`	1.0	String representation of the version of the API implemented by the conforming binding. The value is a dotted decimal of the form `MAJOR.MINOR`; it is `"2.0"` for this revision of the specification.

5.2. API Interfaces

5.2.1. Storage Card

Name	`StorageCard`
Kind	Interface
Namespace	`storagecard.card`
Extends	`SmartCard` (CNA-TR-API)
Since	1.0
Purpose	Represents a storage card and exposes operations to retrieve its memory image and metadata.

Get Block

Signature	`getBlock(blockAddress: Int) → ByteArray?`
Since	1.0
Description	Returns the data block at the specified block address, as known in the memory image of the storage card. `null` if the block has not been read.
Parameters	`blockAddress` — the address of the block to retrieve.
Returns	The non-empty block data, or `null` if the block has not been read.
Pre-conditions	**Argument** — every parameter is non-`null`. **Range** — the block address is within range.
Errors	None.

Get Blocks

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Signature	`getBlocks(`
	`  fromBlockAddress: Int,`
	`  toBlockAddress: Int`
	`) → ByteArray?`

Since	1.0
Description	Returns the data blocks within the specified range from the memory image of the storage card. The returned array contains the blocks in order, from `fromBlockAddress` to `toBlockAddress` (both inclusive). `null` if at least one block of the range has not been read.
Parameters	`fromBlockAddress` — the starting block address (inclusive). `toBlockAddress` — the ending block address (inclusive).
Returns	The non-empty data blocks in the specified range, or `null` if at least one of them has not been read.
Pre-conditions	**Argument** — every parameter is non-`null`. **Range** — `fromBlockAddress` is not greater than `toBlockAddress`, neither is negative, and both stay within the memory image range.
Errors	None.

Get Product Type

Signature	`getProductType() → StorageCardProductType`
Since	1.0
Description	Returns the product type of the storage card.
Returns	A `StorageCardProductType`.
Pre-conditions	None.
Errors	None.

Get System Block

Signature	`getSystemBlock() → ByteArray?`
Since	1.0
Description	Returns the system block when it has been read. The system block contains card-specific metadata and configuration data. This feature is specific to ST25/SRT512 cards. It becomes available once read through `prepareSt25ReadSystemBlock`.
Returns	A non-empty `ByteArray` containing the system block data, or `null` if the system block has not been read yet.
Pre-conditions	**Capability** — the current card type supports system block access.
Errors	None.

Get UID

Signature	`getUid() → ByteArray`
Since	1.0
Description	Returns the unique identifier (UID) of the storage card.
Returns	A non-empty `ByteArray` containing the UID.
Pre-conditions	None.
Errors	None.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5.2.2. Storage Card API Factory

Name	`StorageCardApiFactory`
Kind	Interface
Namespace	`storagecard`
Since	1.0
Purpose	Factory used by the application to obtain instances of the public types provided by the API.

Create Storage Card Selection Extension

Signature	`createStorageCardSelectionExtension(`
	`  productType: StorageCardProductType`
	`) → StorageCardSelectionExtension`

Since	1.0
Description	Returns a new `StorageCardSelectionExtension` dedicated to the targeted product type.
Parameters	`productType` (`StorageCardProductType`) — the targeted storage card product type.
Returns	A `StorageCardSelectionExtension`.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Create Storage Card Transaction Manager

Signature	`createStorageCardTransactionManager(`
	`  reader: CardReader,`
	`  card: StorageCard,`
	`  securitySettings: StorageCardSecuritySettings`
	`) → StorageCardTransactionManager`

Since	1.0
Description	Returns a new `StorageCardTransactionManager` bound to the supplied reader, to the initial card data provided by the selection process and to the supplied security settings. A default `StorageCardSecuritySettings` disables every duration bound.
Parameters	`reader` (`CardReader` (CNA-TR-API)) — the reader through which the card is reached. `card` (`StorageCard`) — the initial card data provided by the selection process. `securitySettings` (`StorageCardSecuritySettings`) — the security settings to be used.
Returns	A `StorageCardTransactionManager`.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

5.2.3. Storage Card Security Settings

Name	`StorageCardSecuritySettings`
Kind	Data
Namespace	`storagecard.transaction`
Since	2.0
Purpose	Carries the security settings of a storage card transaction, built directly by the application. A single instance MAY be shared by every transaction of a terminal, whatever the product type of the card presented.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Name	Type	Default	Since	Description
`readCommandMaxDurations`	`Map<StorageCardProductType, Long>`	`emptyMap()`	2.0	Maximum duration, in microseconds, of the exchange of a read command, for each product type. The bound applies to **each** read command prepared on the `StorageCardTransactionManager`, taken individually. The bound declared for the product type of the card bound to the transaction manager applies; a product type absent from the map is not bounded. Every declared duration is strictly positive. The effective resolution of the measurement depends on the implementation, which limits the detection to the terminals whose measurement chain is fine enough. This bound is a countermeasure against **card emulation**: a generic RFID emulation device does not answer a read command in the same time as the chip of a genuine storage card, so an abnormally long exchange reveals that the answer does not come from the expected product. The bound does not apply to the card selection, nor to the write and authentication commands.

5.2.4. Storage Card Selection Extension

Name	`StorageCardSelectionExtension`
Kind	Interface
Namespace	`storagecard.card`
Extends	`CardSelectionExtension` (CNA-TR-API)
Since	1.0
Purpose	Allows the application to enrich the selection scenario with optional read and authentication commands executed during the selection phase. An instance is obtained via `StorageCardApiFactory.createStorageCardSelectionExtension`.

Prepare MIFARE Classic Authenticate With Key

Signature	`prepareMifareClassicAuthenticateWithKey(`
	`  blockAddress: Int,`
	`  mifareClassicKeyType: MifareClassicKeyType,`
	`  key: ByteArray`
	`) → Self`

Since	1.1
Description	Prepares a MIFARE Classic authentication command using the supplied 6-byte key. This operation is specific to MIFARE Classic cards and MUST be prepared before reading from or writing to protected sectors. The authentication applies to the entire sector containing the supplied block address. Once authenticated, subsequent read and write operations within the same sector MAY be performed without re-authentication, until the card is removed from the field or another sector is accessed. When the key value is supplied this way, it is forwarded to the reader to be stored as a **volatile** key at index 0 (see PC/SC **Load Key** command). The volatile key is temporary and is erased after usage, when the reader is powered off. **Security note:** this operation transmits the key value over the application-reader communication channel. For production environments and security-sensitive applications, it is RECOMMENDED to use the `prepareMifareClassicAuthenticateWithKeyNumber` instead, which references a key already stored in the reader without transmitting its value.
Parameters	`blockAddress` — the address of any block within the sector to authenticate. `mifareClassicKeyType` (`MifareClassicKeyType`) — the type of key to use (Key A or Key B). `key` — the 6-byte key data for authentication.
Returns	The current instance (`Self`).

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Pre-conditions	**Argument**—every parameter is non-`null`. **Range**—the block address is within range. **Argument**—the key is exactly 6 bytes long. **Capability**—the current card type supports authentication.
Errors	None.

Prepare MIFARE Classic Authenticate With Key Number

Signature	`prepareMifareClassicAuthenticateWithKeyNumber(`
	`  blockAddress: Int,`
	`  mifareClassicKeyType: MifareClassicKeyType,`
	`  keyNumber: Int`
	`) → Self`

Since	1.1
Description	Prepares a MIFARE Classic authentication command using a key referenced by its storage index in the reader's key storage. This operation is specific to MIFARE Classic cards and MUST be prepared before reading from or writing to protected sectors. The authentication applies to the entire sector containing the supplied block address. Referencing the key by index allows pre-configured keys to be used without transmitting their value over the communication channel, which provides enhanced security. Once authenticated, subsequent read and write operations within the same sector MAY be performed without re-authentication, until the card is removed from the field or another sector is accessed.
Parameters	`blockAddress` —the address of any block within the sector to authenticate. `mifareClassicKeyType` (`MifareClassicKeyType`)—the type of key to use (Key A or Key B). `keyNumber` —the index of the key in the reader's key storage.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`. **Range**—the block address is within range. **Argument**—the key number is valid. **Capability**—the current card type supports authentication.
Errors	None.

Prepare Read Block

Signature	`prepareReadBlock(blockAddress: Int) → Self`
Since	1.0
Description	Prepares the reading of a specific block. Block addresses start at `0`; the maximum value is `StorageCardProductType.blockCount` - 1. Once processed, the result is available in the `StorageCard` via `getBlock`.
Parameters	`blockAddress`—the address of the block to be read.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`. **Range**—the block address is within range.
Errors	None.
See also	`StorageCardProductType.blockCount`

Prepare Read Blocks

Signature	`prepareReadBlocks(`
	`  fromBlockAddress: Int,`
	`  toBlockAddress: Int`
	`) → Self`

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Since	1.0
Description	Prepares the reading of a range of blocks. Block addresses start at `0`; the maximum value is `StorageCardProductType.blockCount` - 1. Once processed, the result is available in the `StorageCard` via `getBlock` / `getBlocks`.
Parameters	`fromBlockAddress` — the starting block address (inclusive). `toBlockAddress` — the ending block address (inclusive).
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Range** — every argument is within range.
Errors	None.
See also	`StorageCardProductType.blockCount`

Prepare ST25 Read System Block

Signature	`prepareSt25ReadSystemBlock() → Self`
Since	1.2
Description	Prepares the reading of the system block of an ST25/SRT512 storage card. This operation is specific to the ST25 and SRT512 card types, which expose a system block at address 255 carrying card-specific metadata and configuration data. Once processed, the result is available in the `StorageCard` via `getSystemBlock`.
Returns	The current instance (`Self`).
Pre-conditions	**Capability** — the current card type is ST25/SRT512.
Errors	None.
See also	`StorageCard.getSystemBlock`

5.2.5. Storage Card Transaction Manager

Name	`StorageCardTransactionManager`
Kind	Interface
Namespace	`storagecard.transaction`
Extends	`CardTransactionManager` (CNA-TR-API)
Since	1.0
Purpose	Manages APDU exchanges with a storage card. It allows the application to prepare read and write operations, batch them into a single transaction, and control the underlying communication channel. An instance is obtained via `StorageCardApiFactory.createStorageCardTransactionManager`.

The inherited `processCommands()` operation sends every APDU corresponding to a prepared command, in the order in which it was prepared, and interrupts the process at the first failed command.

For read commands, it updates the `StorageCard` memory image with the data retrieved from the card;

for write commands, it updates the memory image with the data written **only** when the write is confirmed successful.

For card technologies that do not provide reliable status codes (e.g. ST25/SRT512), confirmation is obtained via an automatic verification read, and the memory image is updated only if this verification passes.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Prepare MIFARE Classic Authenticate With Key

Signature	`prepareMifareClassicAuthenticateWithKey(`
	`  blockAddress: Int,`
	`  mifareClassicKeyType: MifareClassicKeyType,`
	`  key: ByteArray`
	`) → Self`

Since	1.1
Description	Prepares a MIFARE Classic authentication command using the supplied 6-byte key. This operation is specific to MIFARE Classic cards and MUST be prepared before reading from or writing to protected sectors. The authentication applies to the entire sector containing the supplied block address. Once authenticated, subsequent read and write operations within the same sector MAY be performed without re-authentication, until the card is removed from the field or another sector is accessed. When the key value is supplied this way, it is forwarded to the reader to be stored as a **volatile** key at index 0 (see PC/SC **Load Key** command). The volatile key is temporary and is erased after usage, when the reader is powered off. **Security note:** this operation transmits the key value over the application-reader communication channel. For production environments and security-sensitive applications, it is RECOMMENDED to use the `prepareMifareClassicAuthenticateWithKeyNumber` instead, which references a key already stored in the reader without transmitting its value.
Parameters	`blockAddress` —the address of any block within the sector to authenticate. `mifareClassicKeyType` (`MifareClassicKeyType`) —the type of key to use (Key A or Key B). `key` —the 6-byte key data for authentication.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`. **Range**—the block address is within range. **Argument**—the key is exactly 6 bytes long. **Capability**—the current card type supports authentication.
Errors	None.

Prepare MIFARE Classic Authenticate With Key Number

Signature	`prepareMifareClassicAuthenticateWithKeyNumber(`
	`  blockAddress: Int,`
	`  mifareClassicKeyType: MifareClassicKeyType,`
	`  keyNumber: Int`
	`) → Self`

Since	1.1
Description	Prepares a MIFARE Classic authentication command using a key referenced by its storage index in the reader's key storage. This operation is specific to MIFARE Classic cards and MUST be prepared before reading from or writing to protected sectors. The authentication applies to the entire sector containing the supplied block address. Referencing the key by index allows pre-configured keys to be used without transmitting their value over the communication channel, which provides enhanced security. Once authenticated, subsequent read and write operations within the same sector MAY be performed without re-authentication, until the card is removed from the field or another sector is accessed.
Parameters	`blockAddress` —the address of any block within the sector to authenticate. `mifareClassicKeyType` (`MifareClassicKeyType`) —the type of key to use (Key A or Key B). `keyNumber` —the index of the key in the reader's key storage.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`. **Range**—the block address is within range. **Argument**—the key number is valid. **Capability**—the current card type supports authentication.
Errors	None.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Prepare Read Block

Signature	`prepareReadBlock(blockAddress: Int) → Self`
Since	1.0
Description	Prepares the reading of a specific block. Block addresses start at `0`; the maximum value is `StorageCardProductType.blockCount` - 1. Once processed, the result is available in the `StorageCard` via `getBlock`. When the transaction manager has been created with `StorageCardSecuritySettings` declaring a bound for the product type of the card, each read command exchange is bounded by that duration; an overrun raises `StorageCardInvalidCardResponse`.
Parameters	`blockAddress` — the address of the block to be read.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Range** — the block address is within range.
Errors	None.
See also	`StorageCardProductType.blockCount`

Prepare Read Blocks

Signature	`prepareReadBlocks(`
	`  fromBlockAddress: Int,`
	`  toBlockAddress: Int`
	`) → Self`

Since	1.0
Description	Prepares the reading of a range of blocks. Block addresses start at `0`; the maximum value is `StorageCardProductType.blockCount` - 1. Once processed, the result is available in the `StorageCard` via `getBlock` / `getBlocks`. When the transaction manager has been created with `StorageCardSecuritySettings` declaring a bound for the product type of the card, each read command exchange is bounded by that duration; an overrun raises `StorageCardInvalidCardResponse`.
Parameters	`fromBlockAddress` — the starting block address (inclusive). `toBlockAddress` — the ending block address (inclusive).
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Range** — every argument is within range.
Errors	None.
See also	`StorageCardProductType.blockCount`

Prepare ST25 Read System Block

Signature	`prepareSt25ReadSystemBlock() → Self`
Since	1.1
Description	Prepares the reading of the system block of an ST25/SRT512 storage card. This operation is specific to the ST25 and SRT512 card types, which expose a system block at address 255 carrying card-specific metadata and configuration data. Once processed, the result is available in the `StorageCard` via `getSystemBlock`. When the transaction manager has been created with `StorageCardSecuritySettings` declaring a bound for the product type of the card, each read command exchange is bounded by that duration; an overrun raises `StorageCardInvalidCardResponse`.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Returns	The current instance (`Self`).
Pre-conditions	**Capability**—the current card type is ST25/SRT512.
Errors	None.
See also	`StorageCard.getSystemBlock`

Prepare ST25 Write System Block

Signature	`prepareSt25WriteSystemBlock(`
	`  commandId: Int,`
	`  data: ByteArray`
	`) → Self`

Since	1.1
Description	Prepares the writing of data to the system block of an ST25/SRT512 storage card. This operation is specific to the ST25 and SRT512 card types, which expose a system block at address 255 carrying card-specific metadata and configuration data. The data length MUST match the block size defined by the card's `StorageCardProductType`. **Important:** the application is responsible for ensuring that the write operations are coherent with the target card's technology (e.g. OTP bits). The provided data MUST represent the **expected final state** of the system block after the write. Because ST25/SRT512 cards do not provide reliable status codes, an automatic verification read is performed and the transaction fails if the physical state does not match the provided data. The `commandId` MAY later be obtained via the `commandId` property of the error, if this specific command causes one to be raised.
Parameters	`commandId`—the application-supplied identifier of this command. `data` —the data to be written to the system block (expected final state).
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`. **Argument**—the length of `data` matches the block size. **Capability**—the current card type is ST25/SRT512.
Errors	None.
See also	`StorageCardProductType.blockSize`

Prepare Write Blocks

Signature	`prepareWriteBlocks(`
	`  commandId: Int,`
	`  fromBlockAddress: Int,`
	`  data: ByteArray`
	`) → Self`

Since	1.0
Description	Prepares the writing of one or more blocks starting from the specified block offset. The number of blocks written is determined by the length of `data` divided by the block size of the card. **Important:** the application is responsible for ensuring that the write operations are coherent with the target card's technology (e.g. OTP bits, counters). The provided data MUST represent the **expected final state** of the blocks after the write. For cards that do not provide reliable status codes, an automatic verification read is performed and the transaction fails if the physical state does not match the provided data. The `commandId` MAY later be obtained via the `commandId` property of the error, if this specific command causes one to be raised.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Parameters	`commandId` — the application-supplied identifier of this command. `fromBlockAddress` — the offset at which the blocks are written. `data` — the data to be written (expected final state).
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — the length of `data` is a multiple of the block size.
Errors	None.
See also	`StorageCardProductType.blockCount`

5.3. Enumerations

5.3.1. MIFARE Classic Key Type

Name	`MifareClassicKeyType`
Kind	Enumeration
Namespace	`storagecard`
Since	1.1
Purpose	Enumerates the MIFARE Classic key types used for authentication.

MIFARE Classic cards support two types of keys per sector. Each sector can be protected independently by these keys, allowing fine-grained access control.

Value	Since	Description
`KEY_A`	1.1	Primary key used for authentication to a MIFARE Classic sector. In most configurations, Key A has read/write permissions while Key B may have restricted permissions.
`KEY_B`	1.1	Secondary key used for authentication to a MIFARE Classic sector. Key B is often used for restricted operations or read-only access, depending on the sector's access conditions.

5.3.2. Storage Card Product Type

Name	`StorageCardProductType`
Kind	Enumeration
Namespace	`storagecard.card`
Since	1.0
Purpose	Enumerates the storage card products supported by the API and exposes their structural properties.

Name	Type	Since	Description
`blockCount`	`Int`	1.0	Number of blocks in the main memory area of the card. Additional **system** blocks MAY exist and are not included in this count.
`blockSize`	`Int`	1.0	Size of each block, in bytes.
`hasSystemBlock`	`Boolean`	1.0	Indicates whether the product type exposes an accessible system block. For ST25/SRT512 cards, the system block is accessible at address 255. When a system block is available, it MAY be read using the appropriate `prepare*` operations during selection or transaction processing.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Name	Type	Since	Description
`hasWriteAcknowledgment`	`Boolean`	1.0	Indicates whether the product type provides a reliable acknowledgment after write operations: when `true`, a successful write response guarantees that the data has been correctly stored; when `false`, a verification read is performed to confirm the write.
`hasAuthentication`	`Boolean`	1.1	Indicates whether the product type requires authentication before read/write operations: when `true`, the application MUST authenticate the targeted sector before issuing read or write commands.

Value	`blockCount`	`blockSize`	`hasSystemBlock`	`hasWriteAcknowledgment`	`hasAuthentication`	Since	Description
`MIFARE_ULTRALIGHT`	`16`	`4`	`false`	`true`	`false`	1.0	MIFARE Ultralight.
`MIFARE_CLASSIC_1K`	`64`	`16`	`false`	`true`	`true`	1.1	MIFARE Classic 1K.
`MIFARE_CLASSIC_4K`	`256`	`16`	`false`	`true`	`true`	1.1	MIFARE Classic 4K.
`ST25_SRT512`	`16`	`4`	`true`	`false`	`false`	1.0	ST25 SRT512.

5.4. Errors

Every error defined below carries two properties: `message` (`String`), which describes the condition, and `cause` (`Any?`), the underlying error that triggered it, `null` when there is none. They are not restated for each error.

The storage-card errors—`StorageCardReaderCommunication`, `StorageCardCardCommunication`, `StorageCardInvalidCardResponse` and `StorageCardAuthenticationFailed`—additionally carry `blockAddress` (`Int?`), the block address involved in the error, and `commandId` (`Int?`), the application-supplied identifier of the failing command. Both are `null` when the information is not available.

`StorageCardInvalidCardResponse` is also raised when the duration of a read command exchange exceeds the bound declared for the product type of the card in `StorageCardSecuritySettings`.

5.4.1. Storage Card Authentication Failed

Name	`StorageCardAuthenticationFailed`
Kind	Error
Namespace	`storagecard`
Since	1.1
Purpose	Indicates that an authentication attempt on a storage card has failed (typically due to incorrect key data or key type).

5.4.2. Storage Card Card Communication

Name	`StorageCardCardCommunication`
Kind	Error
Namespace	`storagecard`
Extends	`CardCommunication` (CNA-TR-API)
Since	1.0

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Purpose	Indicates an input/output error during the dialog with the storage card — e.g. transmission failures, card removal during processing, or a failed automatic verification read after a write operation on cards lacking reliable write acknowledgment.

5.4.3. Storage Card Invalid Card Response

Name	`StorageCardInvalidCardResponse`
Kind	Error
Namespace	`storagecard`
Extends	`InvalidCardResponse` (CNA-TR-API)
Since	1.0
Purpose	Indicates that a command returned an unexpected or invalid status while interacting with the storage card.

5.4.4. Storage Card Reader Communication

Name	`StorageCardReaderCommunication`
Kind	Error
Namespace	`storagecard`
Extends	`ReaderCommunication` (CNA-TR-API)
Since	1.0
Purpose	Indicates a low-level reader communication failure (lost connection, hardware malfunction, driver issue) preventing the command from being transmitted to the storage card.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement